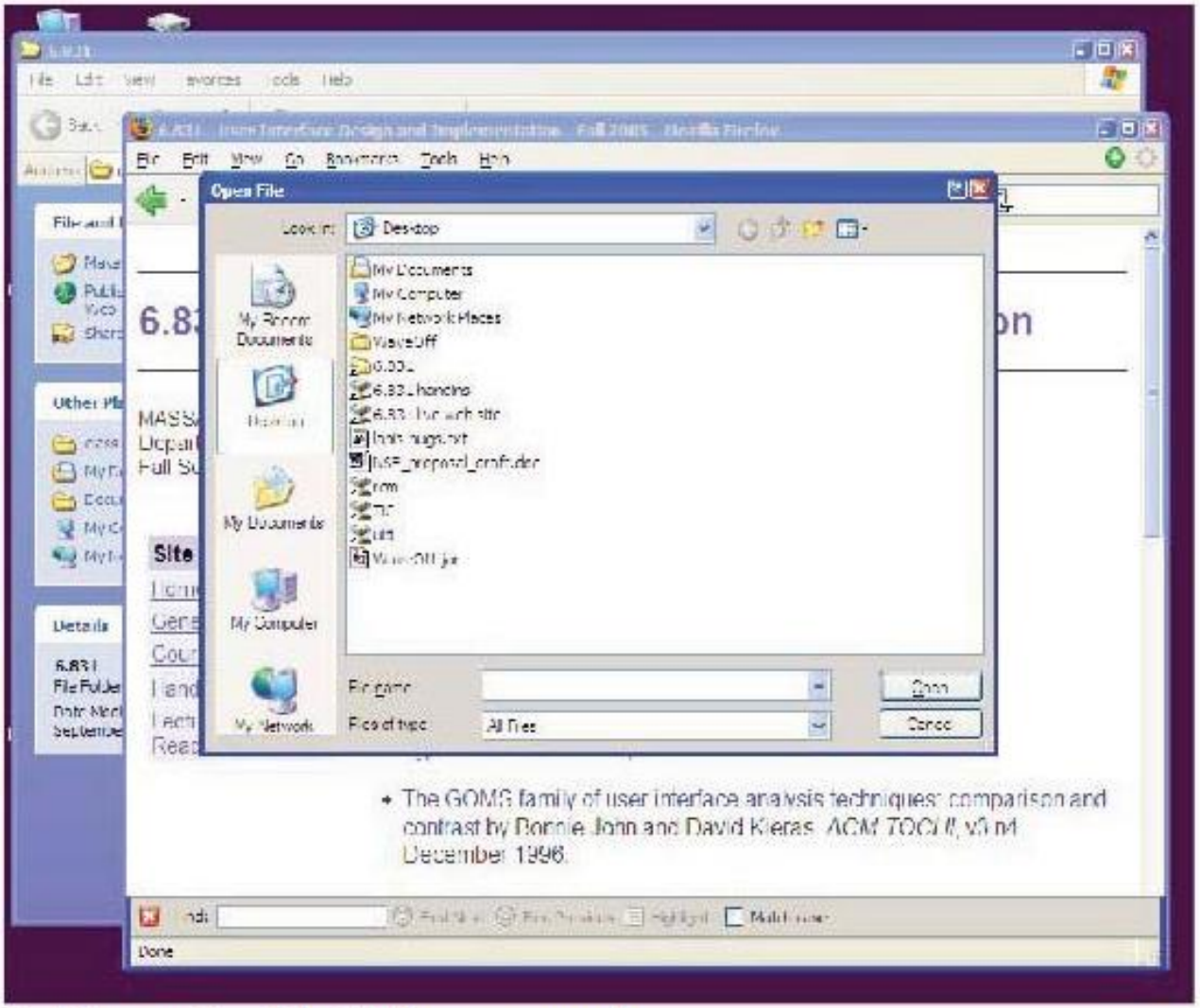
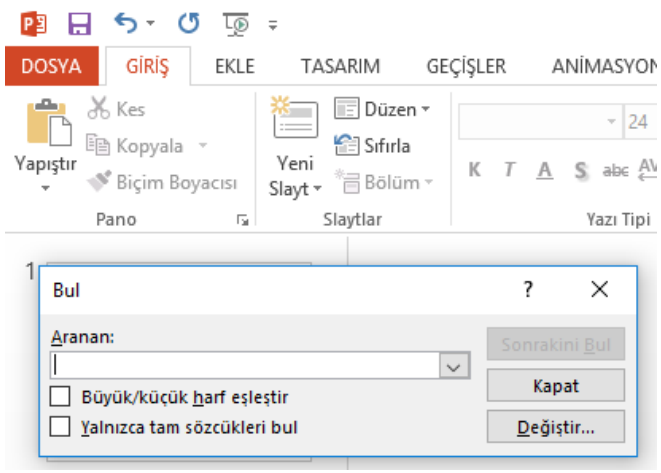
Software Architecture

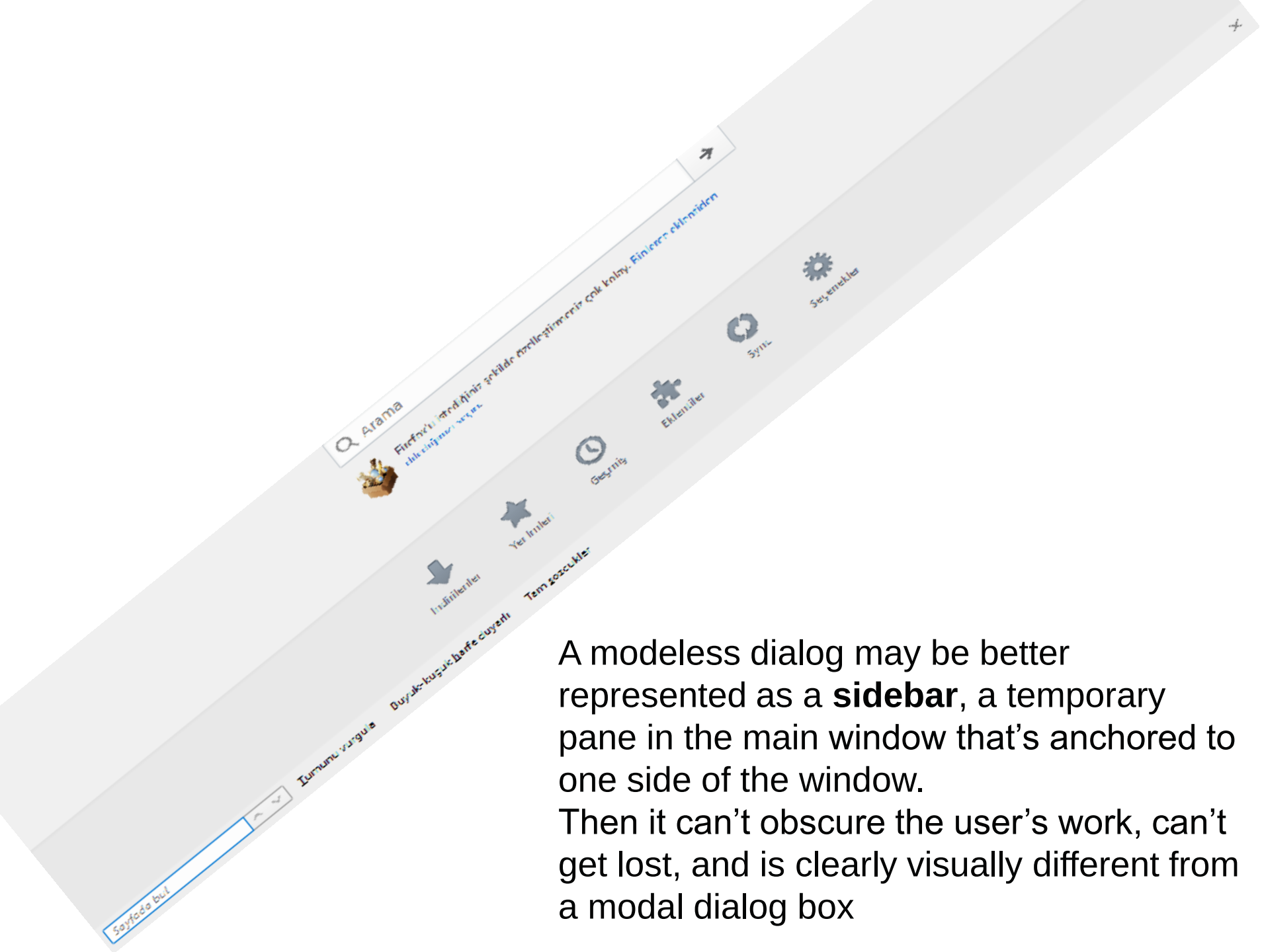
Modal dialog box



© Microsoft. All rights reserved.

- prevents the user from interacting with the application that popped it up
- **error prevention** (modal dialog is always on top, it can't get lost or ignored, and the user can't accidentally change the selection in the main window
- **loss of user control** and reduced **visibility**
- may overload the user's **short-term memory**
- If you try to interact with the main window, Windows gives some nice animated **feedback**
- **consistency**: modal dialogs and modeless dialogs usually look identical



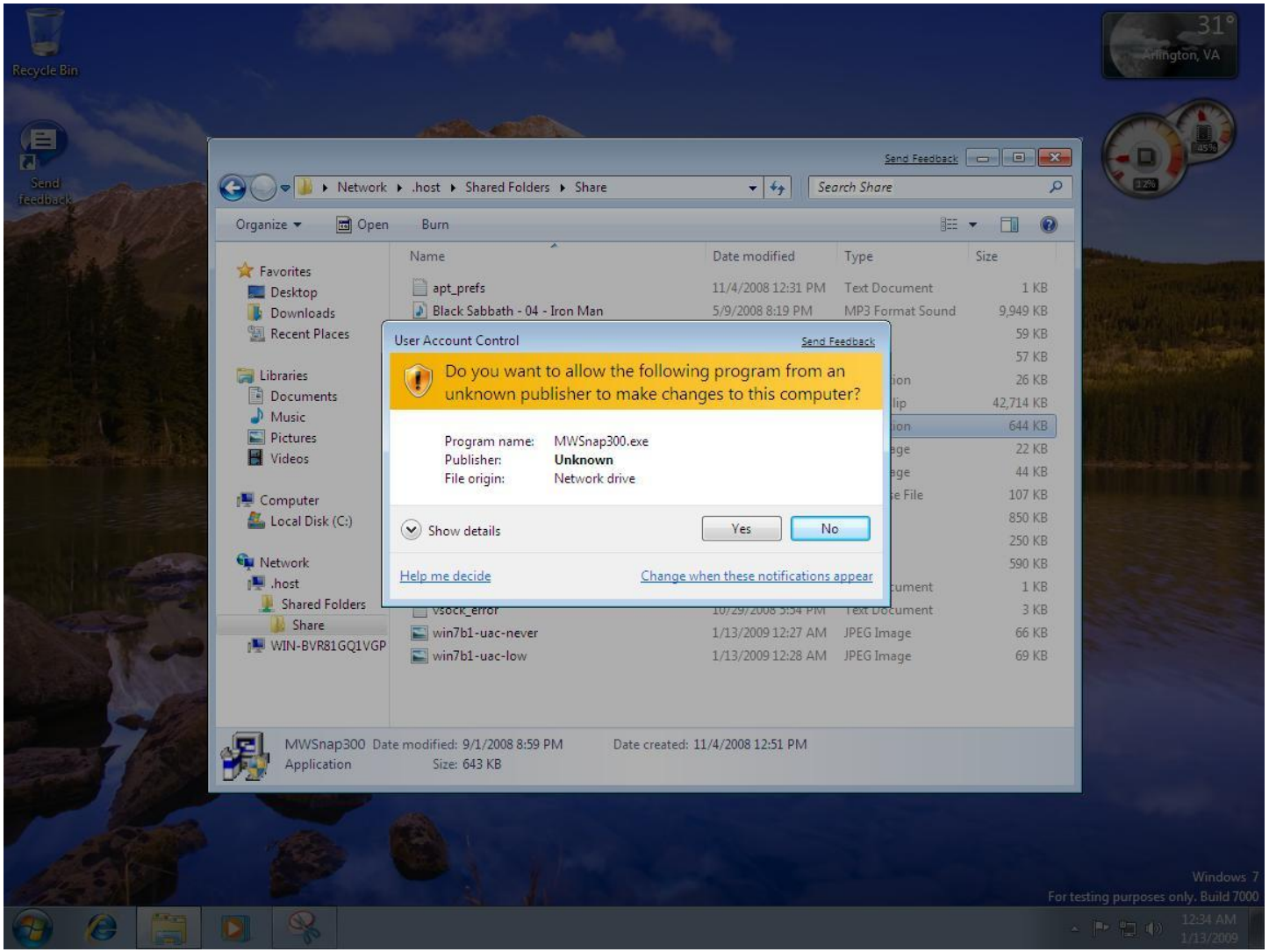


A modeless dialog may be better represented as a **sidebar**, a temporary pane in the main window that’s anchored to one side of the window.
Then it can’t obscure the user’s work, can’t get lost, and is clearly visually different from a modal dialog box

© Apple, Inc. All rights reserved.

- Windows modal dialogs are generally ***application-modal*** – all windows in the application stop responding until the dialog is dismissed
- old GUIs had ***system-modal*** dialogs, which suspended *all* applications
- Mac OS X has a neat improvement, ***window-modal*** dialogs, which are displayed as translucent sheets attached to the titlebar of the blocked window. This tightly associates the dialog with its window, gives a little visibility of what's underneath it in the main window – and allows you to interact with other windows, even if they're from the same application

system-modal dialog



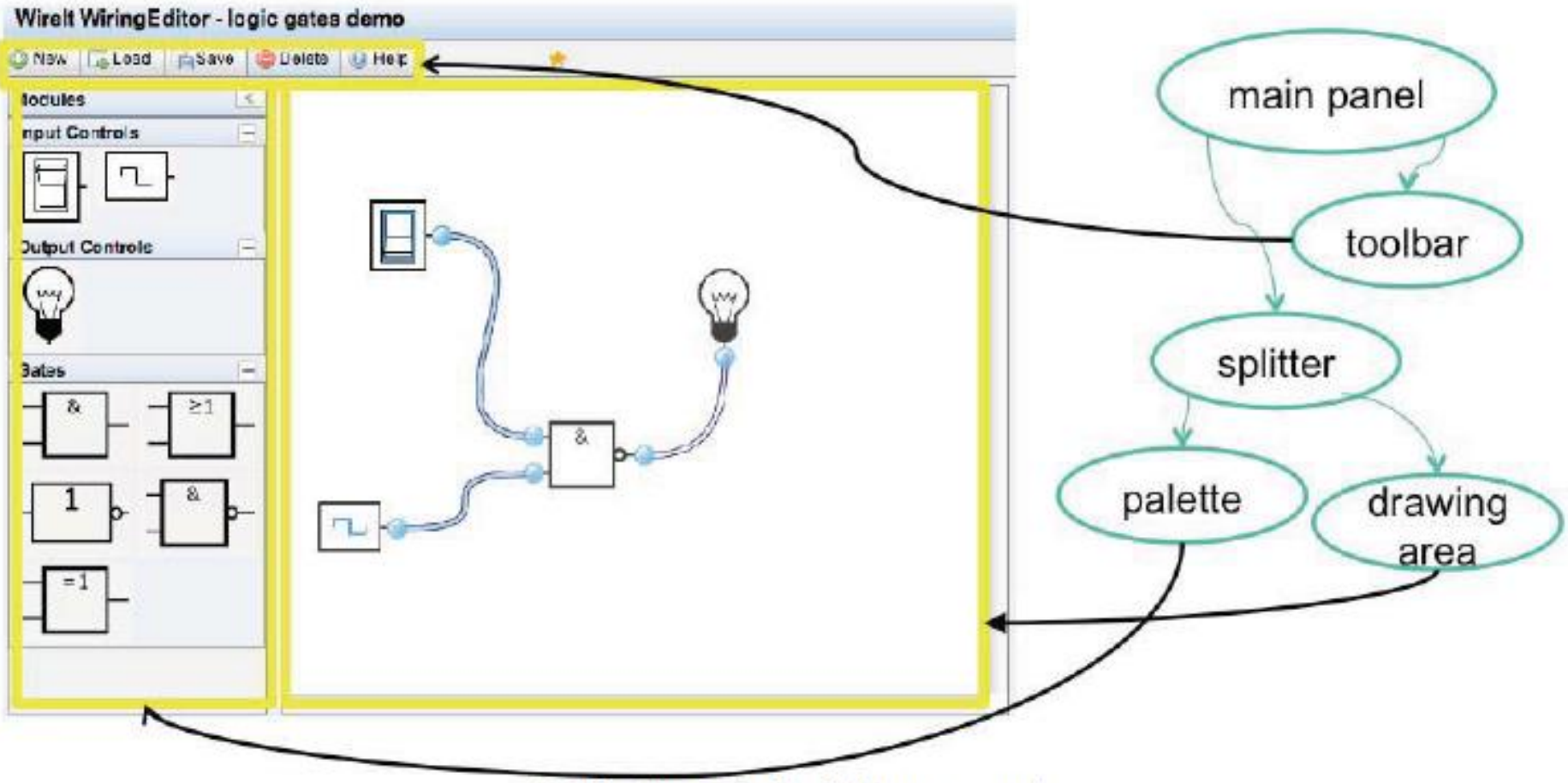
Today's Topics

- Design patterns for GUIs
  - View tree
  - Listener
  - Widget
  - Model-view-controller
- Approaches to GUI programming
  - Procedural
  - Declarative
  - Direct manipulation
- Web UI at lightning speed
  - HTML
  - Javascript
  - jQuery

backend development of web applications falls outside the scope of the course material in this class

View Tree

- A GUI is structured as a tree of views
  - A view is an object that displays itself on a region of the screen



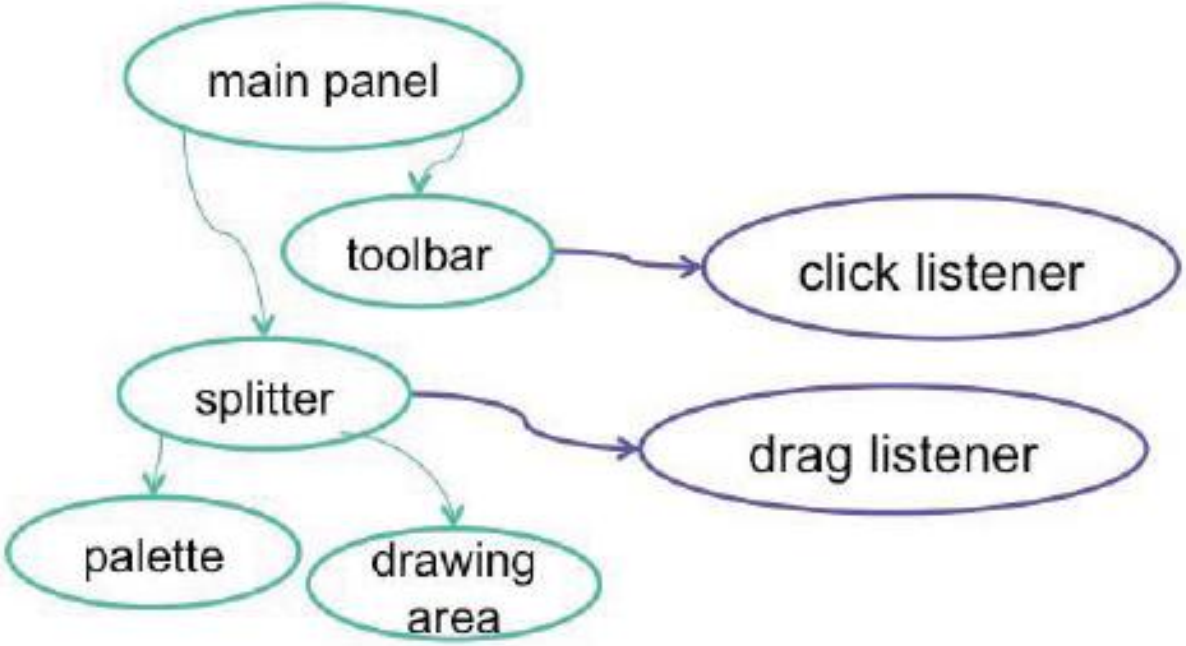
© Eric Abouaf. All rights reserved.

How the View Tree Is Used

- Output
  - GUIs change their output by **mutating** the view tree
  - A redraw algorithm automatically redraws the affected views
- Input
  - GUIs receive keyboard and mouse input by attaching listeners to views (more on this in a bit)
- Layout
  - Automatic layout algorithm traverses the tree to calculate positions and sizes of views

Input Handling

- Input handlers are associated with views
  - Also called **listeners**, event handlers, subscribers, observers



Listener Pattern

- GUI input handling is an example of the Listener pattern
  - aka Publish-Subscribe, Event, Observer
- An event source generates a stream of discrete events
  - e.g., mouse events
- Listeners register interest in events from the source
  - Can often register only for specific events – e.g., only want mouse events occurring inside a view's bounds
  - Listeners can unsubscribe when they no longer want events
- When an event occurs, the event source distributes it to all interested listeners

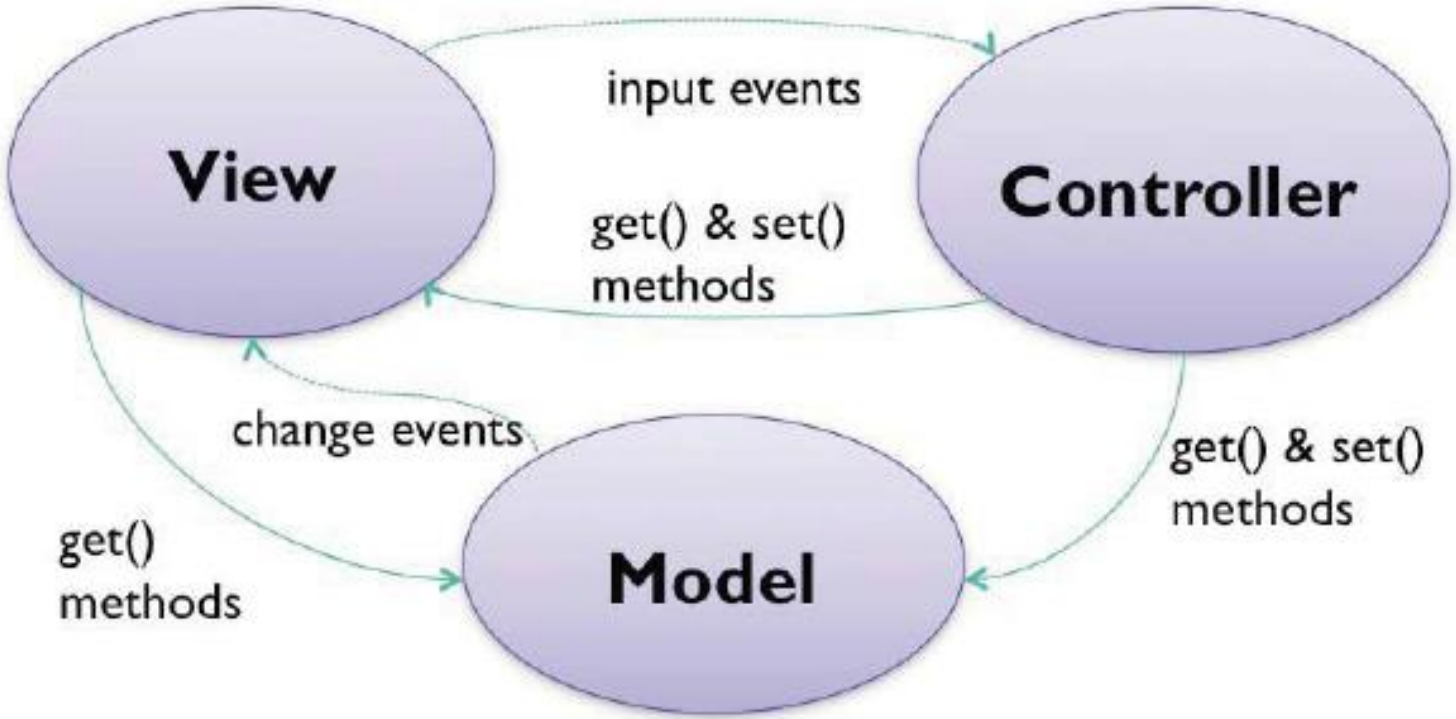
Separating Frontend from Backend

- We've seen how to separate input and output in GUIs
  - Output is represented by the view tree
  - Input is handled by listeners attached to views
- Missing piece is the backend of the system
  - Backend (aka **model**) represents the actual data that the user interface is showing and editing
  - Why do we want to separate this from the user interface?

Model-View-Controller Pattern

View handles output
- gets data from the model to display it
- listens for model changes and updates display

Controller handles input
- listens for input events on the view tre
- calls mutators on model or view



Model maintains application state
- implements state-changing behavior
- sends change events to views

Advantages of Model-View-Controller

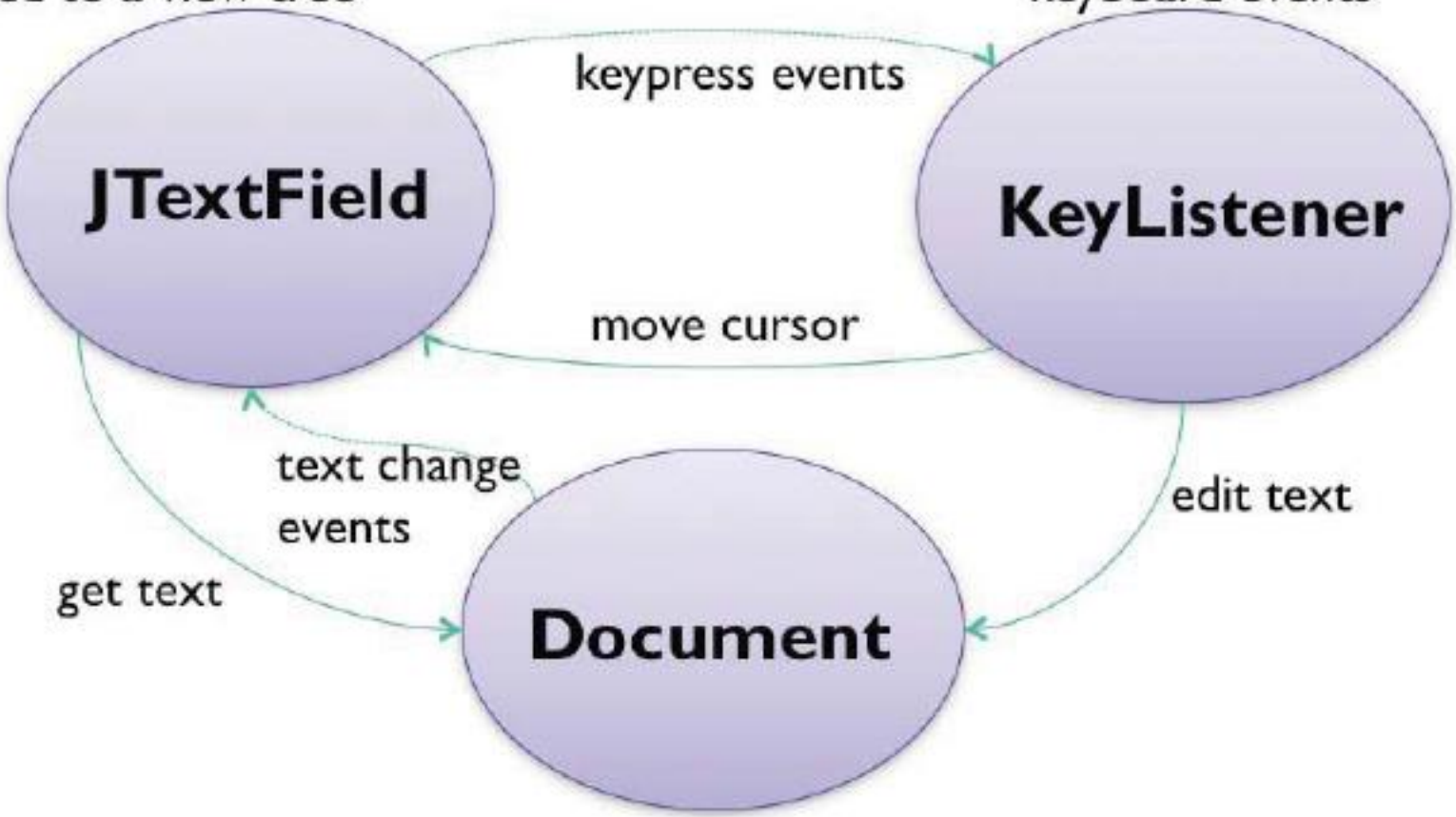
- Separation of responsibilities
  - Each module is responsible for just one feature
    - Model: data
    - View: output
    - Controller: input
- Decoupling
  - View and model are decoupled from each other, so they can be changed independently
  - Model can be reused with other views
  - Multiple views can simultaneously share the same model
  - Views can be reused for other models, as long as the model implements an interface

enables the creation of user interface **toolkits**, which are libraries of reusable interface objects

A Small MVC Example: Textbox

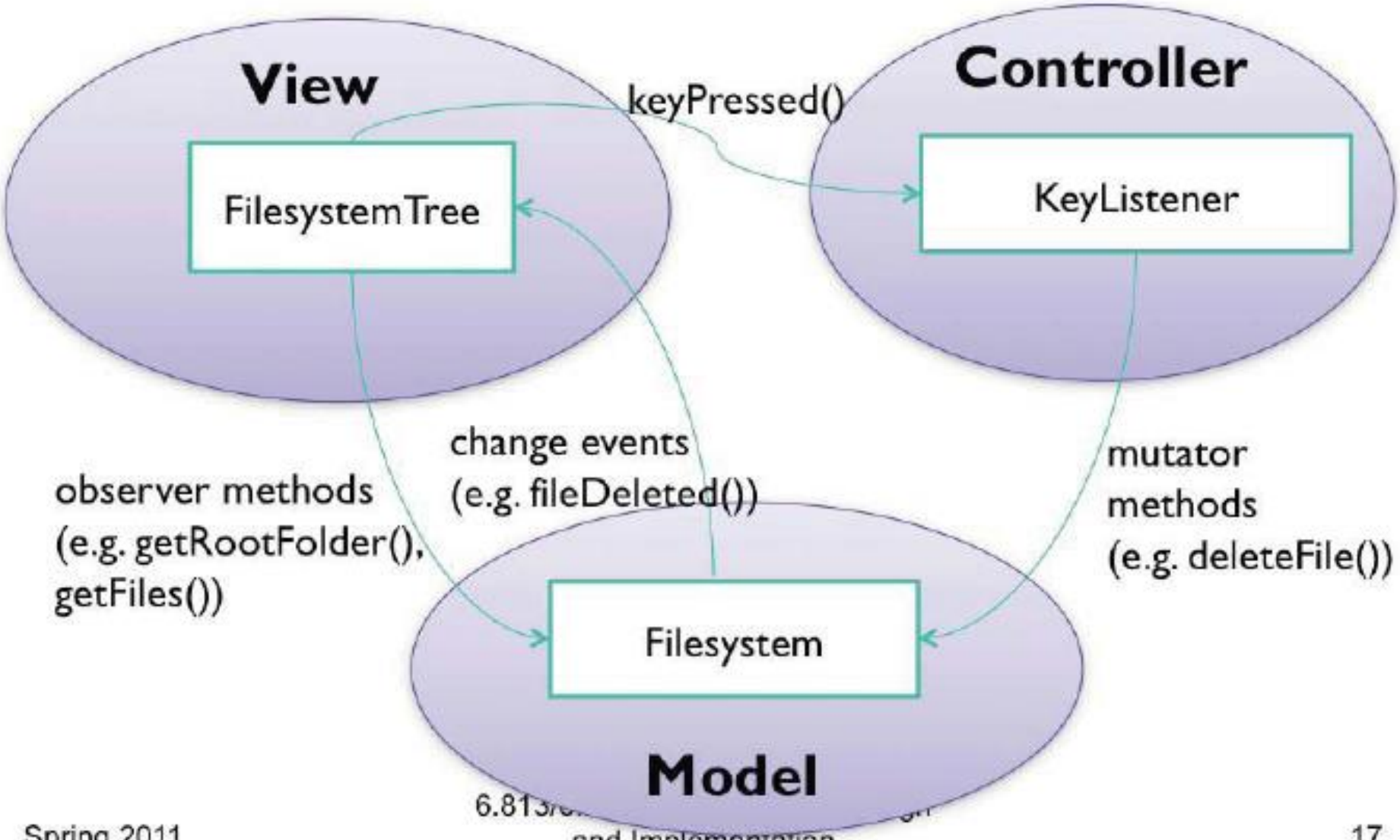
JTextField is a Component that can be added to a view tree

KeyListener is a listener for keyboard events



Document represents a mutable string of characters

A Larger MVC Example



Hard to Separate Controller and View

- Controller often needs output
  - View must provide **affordances** for controller (e.g. scrollbar thumb)
  - View must also provide **feedback** about controller state (e.g., depressed button)
- State shared between controller and view: Who manages the selection?
  - Must be displayed by the view (as blinking text cursor or highlight)
  - Must be updated and used by the controller
  - Should selection be in model?
    - Generally not
    - Some views need independent selections (e.g. two windows on the same document)
    - Other views need synchronized selections (e.g. table view & chart view)

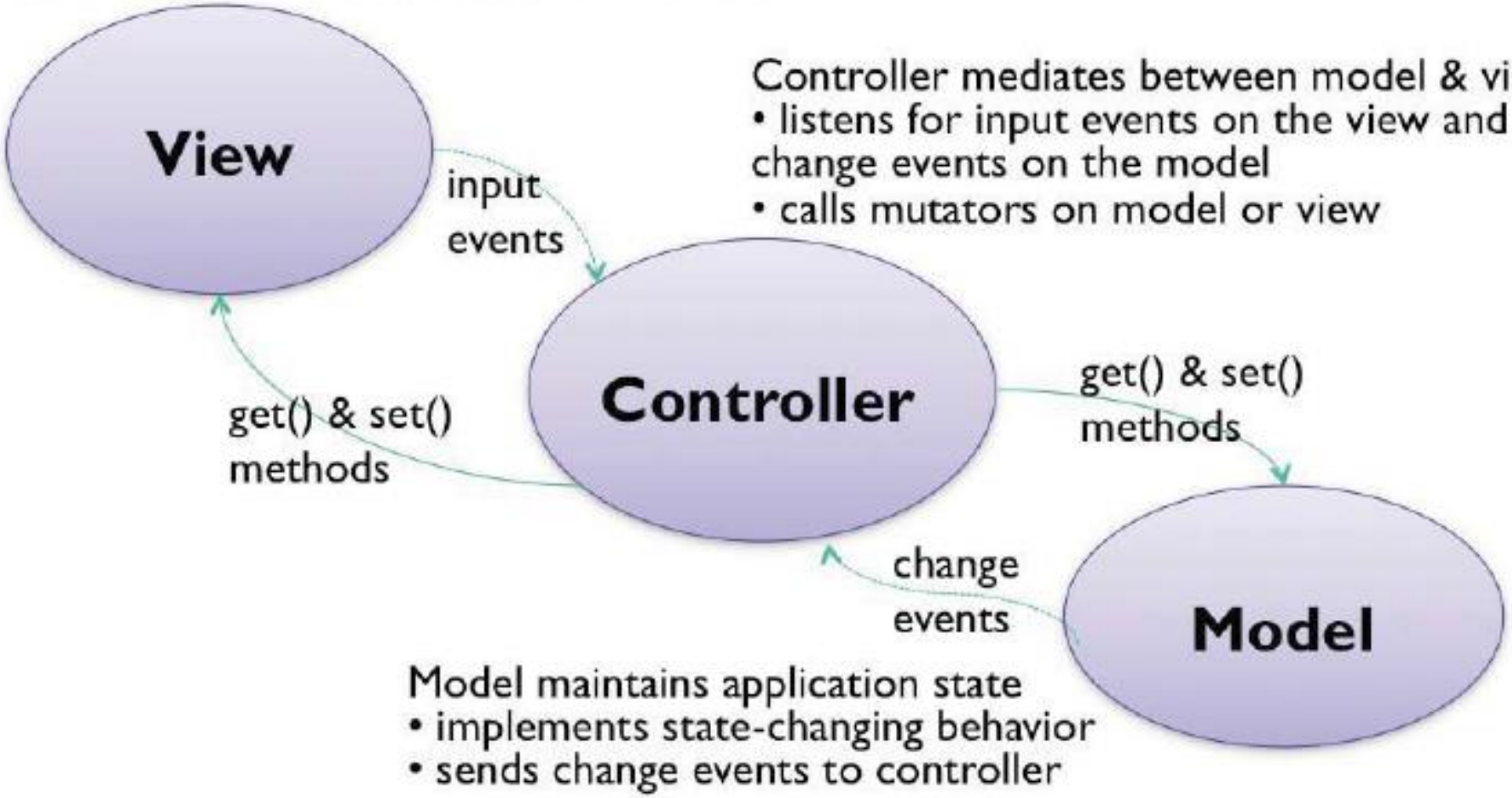
Widget: Tightly Coupled View & Controller

- The MVC idea has largely been superseded by an MV (Model-View) idea
- A widget is a reusable view object that manages both its output and its input
  - Widgets are sometimes called components (Java, Flex) or controls (Windows)
- Examples: scrollbar, button, menubar

view and controller aren’t independently reusable

A Different Perspective on MVC

View handles output & low-level input
- sends high-level events to the controller



Controller mediates between model & view
- listens for input events on the view and change events on the model
- calls mutators on model or view

Model maintains application state
- implements state-changing behavior
- sends change events to controller

GUI Implementation Approaches

- Procedural programming
  - Code that says *how* to get what you want (flow of control)
- Declarative programming
  - Code that says *what* you want (no explicit flow of control)
- Direct manipulation
  - Creating what you want in a direct manipulation interface

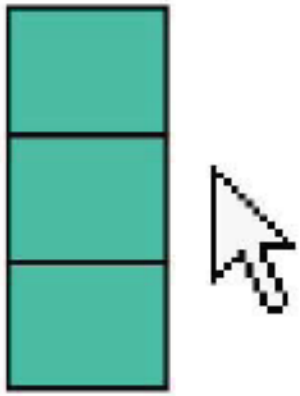
Procedural

1. Put down block A.
2. Put block B on block A.
3. Put block C on block B.

Declarative

A tower of 3 blocks.

Direct Manipulation



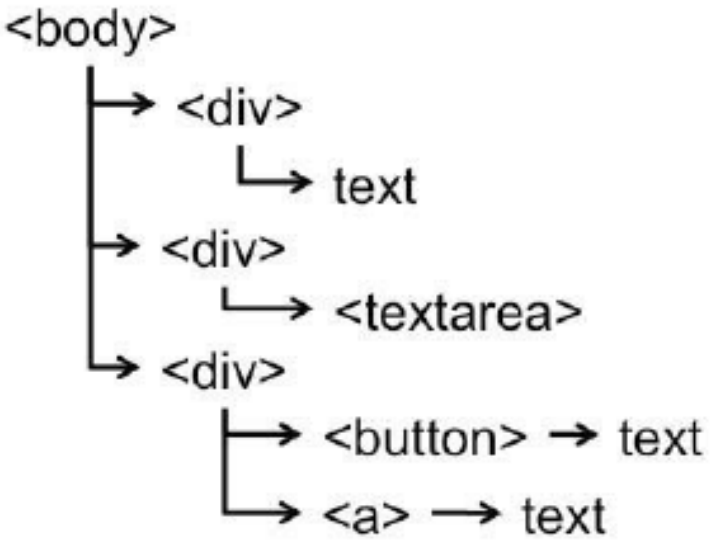
Markup Languages

- HTML **declaratively** specifies a view tree

```
<body>
  <div>What are you doing now?</div>
  <div><textarea></textarea></div>
  <div><button>Send</button> <a href="#">sign out</
   a></div>
</body>
```

What are you doing now?

Send sign out



HTML Syntax

```
<!DOCTYPE html>
<html>
<head></head>
<body>
    <button disabled="true">
        <img src='dont.png' />
        Don't Press Me
    </button>
            <!--
                <button>
                    Press Me Instead
                </button>
            -->
</body>
</html>
```

<, >, and & need to be escaped: < > & respectively

e.g.
http://lmgtfy.com/?l=1&q=rick+roll

Important HTML Elements for UI Design

- **Layout**

Box	
Grid	<table>, <tr>, <td>

- **Text**

Font & color	

- **Widgets**

Hyperlink	<a>
Button	<button>
Textbox	<input type="text">
Multiline text	<textarea>
Rich text	
Drop-down	<select> <option>
Listbox	<select multiple="true">
Checkbox	<input type="checkbox">
Radiobutton	<input type="radio">

- **Pixel output**
  <img>

- **Stroke output**
  <canvas> (Firefox, Safari)

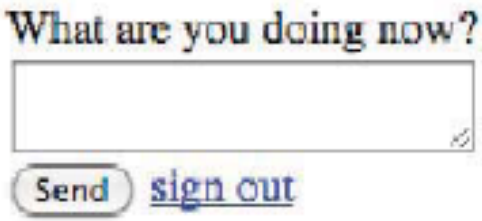
- **Javascript code**
  <script>

- **CSS style sheets**
  <style>

View Tree Manipulation

- Javascript can **procedurally** mutate a view tree

```
<script>
var doc = document
var div1 = doc.createElement("div")
 div1.appendChild(doc.createTextNode("What are you doing now?")
…
var div3 = doc.createElement("div")
 var button = doc.createElement("button")
  button.appendChild(doc.createTextNode("Send"))
  div3.appendChild(button)
 var a = doc.createElement("a")
  a.setAttribute("href", "#")
  a.appendChild(doc.createTextNode("sign out"))
  div3.appendChild(a)
</script>
```



Document Object Model (DOM) is a standard set of classes and methods for interacting with a tree of HTML or XML objects procedurally

Javascript in One Slide

Like Java…

expressions

```
hyp = Math.sqrt(a*a + b*b)
console.log("Hello"
            + ", world");
```

statements

```
if (a < b) { return a }
   else { return b }
```

comments

```
/* this
    is a comment */
// and so is this
```

Like Python…

no declared types

```
var x = 5;
for (var i = 0; i < 10; ++i) {…}
```

objects and arrays are dynamic

```
var obj = { x: 5, y: -1 };
obj.z = 8;
var list = ["a", "b"];
list[2] = "c";
```

functions are first-class

```
function square(x) { return x*x; }
var double = function(a) {
                 return 2*a; }
```

jQuery in One Slide

- Select nodes

```
$("#send")
$(".toolbar")
$("button")
```

```
<button id="send" class="toolbar">
  Send
</button>
```

- Create nodes

```
$('<button class="toolbar"></button>')
```

- Act on nodes

```
$("#send").text()                // returns "Send"
$("#send").text("Tweet")         // changes button label
$(".toolbar").attr("disabled", "true")
$("#send").click(function() { … })
$("#textarea").val()
$("#mainPanel").html("<button>Press Me</button>")
```

Mixing Declarative and Procedural Code

```
<body>
  <div>What are you doing now?</div>
  <div><textarea id="msg"></textarea></div>
  <div><button id="send">Send</button></div>
  <div id="sent" style="font-style: italic">
    <div>Sent messages appear here.</div>
  </div>
</body>
```

What are you doing now?

Send

Sent messages appear here.

template is constructed as a string of characters

```
<script src="http://code.jquery.com/jquery-1.5.min.js"></script>
<script>
  $(function() {
    $("#send").click(function() {
      var msg = $("#msg").val()

    })
  })
</script>
```

```
var sent = $("#sent").html()
sent += "<div>" + msg + "</div>"
$("#sent").html(sent)
```

```
var div = $("<div></div>").text(msg)
$("#sent").append(div)
```

as a data structure of objects

Shorthand for $(document).ready(function() {…})), an event handler attached to the root of the view tree (the *document*). This is called once, after the entire HTML has been parsed and the tree has been constructed

```
<!DOCTYPE html>
<html>
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>

<script>
$(document).ready(function(){
    $("p").click(function(){
        $(this).hide();
    });
});

$(function(){
        $("#send").click(function(){
        var msg = $("#msg").val()
        var div = $("<div></div>").text(msg)
        //$("#send").append(div)
        $("#sent").append(div)
    })
})
</script>
</head>
<body>
```

```
<p>If you click on me, I will disappear.</p>
<p>Click me away!</p>
<p>Click me too!</p>
<div><textarea
id="msg"></textarea></div>
<div><button
id="send">Send</button></div>
<div id="sent"><div>Sent messages
appear here</div></div>

</body>
</html>
```

If you click on me, I will disappear.

Click me away!

Click me too!

a

Send

Sent messages appear here

a

Advantages & Disadvantages of Declarative UI

- Usually more compact
- Programmer only has to know how to say *what*, not *how*
  - Automatic algorithms are responsible for figuring out how
- May be harder to debug
  - Can't set breakpoints, single-step, print in a declarative specification
  - Debugging may be more trial-and-error
- Authoring tools are possible
  - Declarative spec can be loaded and saved by a tool; procedural specs generally can't

allow the user interface to be constructed by direct manipulation rather than coding

Summary

- Design patterns
  - View tree is the primary structuring pattern for GUIs, used for output, input, and layout
  - Listener is used for input and model-view communication
  - Model-view-controller decouples backend from GUI
- Approaches to GUI programming
  - Procedural, declarative, direct manipulation
  - HTML, Javascript, jQuery